

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу
«Операционные системы»

Группа: М8О-211БВ-24

Студент: Отмахов Н.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 11.11.2025

Москва, 2025

Постановка задачи

Вариант 12.

Пользователь вводит команды вида: «число число число». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс производит деление первого числа, на последующие, а результат выводит в файл. Если происходит деление на 0, то тогда дочерний и родительский процесс завершают свою работу. Проверка деления на 0 должна осуществляться на стороне дочернего процесса. Числа имеют тип `int`. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

Системные вызовы для работы с разделяемой памятью

- `int shm_open(const char *name, int oflag, mode_t mode)`
Создает или открывает объект разделяемой памяти. Возвращает файловый дескриптор или -1 при ошибке.
- `int ftruncate(int fd, off_t length)`
Изменяет размер файла или разделяемой памяти. Возвращает 0 при успехе, -1 при ошибке.
- `void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset)`
Отображает файл или разделяемую память в адресное пространство процесса. Возвращает указатель на отображенную область или `MAP_FAILED`.
- `int munmap(void *addr, size_t length)`
Удаляет отображение памяти. Возвращает 0 при успехе, -1 при ошибке.
- `int shm_unlink(const char *name)`
Удаляет объект разделяемой памяти. Возвращает 0 при успехе, -1 при ошибке.

Системные вызовы для работы с семафорами

- `sem_t *sem_open(const char *name, int oflag, ...)`
Открывает или создает именованный семафор. Возвращает указатель на семафор или `SEM_FAILED`.
- `int sem_wait(sem_t *sem)`
Ожидает семафор (уменьшает значение на 1). Возвращает 0 при успехе, -1 при ошибке.
- `int sem_post(sem_t *sem)`
Освобождает семафор (увеличивает значение на 1). Возвращает 0 при успехе, -1 при ошибке.
- `int sem_close(sem_t *sem)`
Закрывает семафор. Возвращает 0 при успехе, -1 при ошибке.
- `int sem_unlink(const char *name)`
Удаляет именованный семафор. Возвращает 0 при успехе, -1 при ошибке.

Системные вызовы для управления процессами

- `pid_t fork(void)`
Создает новый процесс-потомок. Возвращает 0 в потомке, PID потомка в родителе, -1 при ошибке.

- `int execv(const char *path, char *const argv[])`
Заменяет текущий процесс новым процессом. Возвращает -1 только при ошибке.
- `pid_t waitpid(pid_t pid, int *wstatus, int options)`
Ожидает завершения указанного процесса. Возвращает PID завершенного процесса или -1.

Функции для работы с файлами и вводом-выводом

- `ssize_t write(int fd, const void *buf, size_t count)`
Записывает данные в файловый дескриптор. Возвращает количество записанных байт или -1.
- `int close(int fd)`
Закрывает файловый дескриптор. Возвращает 0 при успехе, -1 при ошибке.

В ходе лабораторной работы я создавал объекты разделяемой памяти, отображал их в адресное пространство процессов. Также я создавал несколько процессов, чтобы они могли общаться между собой благодаря той самой разделяемой памяти для обработки входных данных. Чтобы не было гонки данных, я использовал семафор.

Код программы

client.c

```
#include <fcntl.h>

#include <stdint.h>

#include <stdbool.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/mman.h>

#include <sys/wait.h>

#include <semaphore.h>

#include <stdio.h>

#include <string.h>

#define SHM_SIZE 4096

int main(int argc, char *argv[]) {

    if (argc != 2) {

        const char msg[] = "usage: ./client <filename>";

        write(STDERR_FILENO, msg, sizeof(msg) - 1);

        exit(EXIT_FAILURE);
```

```
}
```

```
char shm_name[64], sem_server_name[64], sem_child_name[64];
```

```
pid_t mypid = getpid();
```

```
snprintf(shm_name, sizeof(shm_name), "/myapp-%d-shm", mypid);
```

```
snprintf(sem_server_name, sizeof(sem_server_name), "/myapp-%d-srv", mypid);
```

```
snprintf(sem_child_name, sizeof(sem_child_name), "/myapp-%d-chd", mypid);
```

```
// Создаём SHM
```

```
int shm_fd = shm_open(shm_name, O_RDWR | O_CREAT | O_EXCL, 0600);
```

```
if (shm_fd == -1) {
```

```
    const char msg[] = "error: can't open shm\n";
```

```
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
```

```
    exit(EXIT_FAILURE);
```

```
}
```

```
if (ftruncate(shm_fd, SHM_SIZE) == -1) {
```

```
    const char msg[] = "error: can't resize shm\n";
```

```
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
```

```
    exit(EXIT_FAILURE);
```

```
}
```

```
char *shm_buf = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd,  
0);
```

```
if (shm_buf == MAP_FAILED) {
```

```
    const char msg[] = "error: mmap failed\n";
```

```
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
```

```
    exit(EXIT_FAILURE);
```

```
}
```

```
sem_t *sem_server = sem_open(sem_server_name, O_CREAT | O_EXCL, 0600, 1);  
if (sem_server == SEM_FAILED) {  
    const char msg[] = "error: can't create sem\n";  
    write(STDERR_FILENO, msg, sizeof(msg) - 1);  
    exit(EXIT_FAILURE);  
}
```

```
sem_t *sem_child = sem_open(sem_child_name, O_CREAT | O_EXCL, 0600, 0);  
if (sem_child == SEM_FAILED) {  
    const char msg[] = "error: can't create sem\n";  
    write(STDERR_FILENO, msg, sizeof(msg) - 1);  
    exit(EXIT_FAILURE);  
}
```

```
pid_t child = fork();  
if (child == -1) {  
    const char msg[] = "error: can't create child\n";  
    write(STDERR_FILENO, msg, sizeof(msg) - 1);  
    exit(EXIT_FAILURE);  
}
```

```
if (child == 0) {  
    // Дочерний процесс  
    char *args[] = {  
        "./child",  
        argv[1],  
        shm_name,  
        sem_server_name,  
        sem_child_name,  
        NULL
```

```

};

int32_t status = execv("./child", args);
if (status == -1)
{
    const char msg[] = "error: can't execv\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    _exit(EXIT_FAILURE);
}
}

//Родительский процесс
{
    char buf[128];
    snprintf(buf, 128, "%d: I'm parent, child PID = %d\n", (int)mypid, (int)child);
    write(STDOUT_FILENO, buf, strlen(buf));
    const char msg[] = "Enter numbers (like '10 2 5'), blank line to exit.\n";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);
}

bool running = true;
uint32_t *len_ptr = (uint32_t *)shm_buf;
char *text = shm_buf + sizeof(uint32_t);
const size_t max_text_len = SHM_SIZE - sizeof(uint32_t) - 1;

while (running) {
    sem_wait(sem_server);

    if (*len_ptr == UINT32_MAX) {
        const char msg[] = "Child requested termination\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    }
}

```

```
    running = false;
    break;
}
```

```
if (*len_ptr > 0) {
    write(STDOUT_FILENO, "Result: ", 8);
    write(STDOUT_FILENO, text, *len_ptr);
    *len_ptr = 0;
    sem_post(sem_server);
    continue;
}
```

```
write(STDOUT_FILENO, "Input> ", 7);
```

```
char input[max_text_len + 2];
if (fgets(input, sizeof(input), stdin) == NULL) {
    *len_ptr = UINT32_MAX;
    sem_post(sem_child);
    break;
}
```

```
size_t input_len = strlen(input);
if (input_len == 0 || (input_len == 1 && input[0] == '\n')) {

    *len_ptr = UINT32_MAX;
    sem_post(sem_child);
    break;
}
```

```
if (input[input_len - 1] == '\n') {
    input[--input_len] = '\0';
```

```
}
```

```
if (input_len == 0) {  
    *len_ptr = UINT32_MAX;  
    sem_post(sem_child);  
    break;  
}
```

```
if (input_len > max_text_len) {  
    const char msg[] = "Input too long!\n";  
    write(STDERR_FILENO, msg, sizeof(msg) - 1);  
    continue;  
}
```

```
*len_ptr = (uint32_t)input_len;  
memcpy(text, input, input_len);  
text[input_len] = '\0';
```

```
    sem_post(sem_child);  
}
```

```
waitpid(child, NULL, 0);
```

```
sem_close(sem_server);  
sem_close(sem_child);  
sem_unlink(sem_server_name);  
sem_unlink(sem_child_name);
```

```
munmap(shm_buf, SHM_SIZE);  
close(shm_fd);
```



```
shm_unlink(shm_name);
```

```
return 0;
```

```
}
```

child.c

```
#include <stdint.h>
```

```
#include <stdbool.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <string.h>
```

```
#include <fcntl.h>
```

```
#include <sys/mman.h>
```

```
#include <semaphore.h>
```

```
#define SHM_SIZE 4096
```

```
int main(int argc, char **argv) {
```

```
    if (argc != 5) {
```

```
        const char msg[] = "Usage: child <file> <shm> <sem_srv> <sem_chd>\n";
```

```
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
```

```
        _exit(EXIT_FAILURE);
```

```
    }
```

```
    int fd = open(argv[1], O_WRONLY | O_CREAT | O_APPEND, 0600);
```

```
    if (fd == -1) {
```

```
        const char msg[] = "Can't open file\n";
```

```
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
```

```
        _exit(EXIT_FAILURE);
```

```
    }
```

```

int shm_fd = shm_open(argv[2], O_RDWR, 0);

if (shm_fd == -1) {
    const char msg[] = "Can't open shm\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    _exit(EXIT_FAILURE);
}

char *shm_buf = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd,
0);

if (shm_buf == MAP_FAILED) {
    const char msg[] = "Can't mmap\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    _exit(EXIT_FAILURE);
}

sem_t *sem_child = sem_open(argv[4], 0);

if (sem_child == SEM_FAILED) {
    const char msg[] = "Can't create sem\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    _exit(EXIT_FAILURE);
}

sem_t *sem_server = sem_open(argv[3], 0);

if (sem_server == SEM_FAILED) {
    const char msg[] = "Can't create sem\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    _exit(EXIT_FAILURE);
}

uint32_t *len_ptr = (uint32_t *)shm_buf;
char *text = shm_buf + sizeof(uint32_t);

```

```

while (1) {
    sem_wait(sem_child);

    if (*len_ptr == UINT32_MAX) {
        break;
    }

    if (*len_ptr == 0) {
        sem_post(sem_server);
        continue;
    }

    text[*len_ptr] = '\0';

    char *buf = strdup(text);
    if (!buf) {
        const char msg[] = "Out of memory\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        break;
    }

    int nums[100];
    int count = 0;
    char *token = strtok(buf, " \t");

    while (token && count < 100) {
        char *end;
        long val = strtol(token, &end, 10);
        if (*end != '\0') {
            break;
        }
    }
}

```

```
    nums[count++] = (int)val;
    token = strtok(NULL, " \t");
}
```

```
free(buf);
```

```
if (count < 2) {
    const char msg[] = "Need at least two integers\n";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    *len_ptr = 0;
    sem_post(sem_server);
    continue;
}
```

```
for (int i = 1; i < count; i++) {
    if (nums[i] == 0) {
        const char err[] = "Division by zero! Terminating.\n";
        write(STDERR_FILENO, err, sizeof(err) - 1);
        *len_ptr = UINT32_MAX;
        sem_post(sem_server);
        close(fd);
        _exit(EXIT_FAILURE);
    }
}
```

```
int first = nums[0];
char result_buf[1024] = {0};
int total_len = 0;
```

```
for (int i = 1; i < count; i++) {
    int res = first / nums[i];
```

```

int n = snprintf(result_buf + total_len,
                 sizeof(result_buf) - total_len,
                 "%d / %d = %d\n", first, nums[i], res);
if (n < 0 || (size_t)(n + total_len) >= sizeof(result_buf)) break;
total_len += n;

char line[64];
int len = snprintf(line, sizeof(line), "%d / %d = %d\n", first, nums[i], res);
if (len > 0) {
    write(fd, line, len);
}
}

if (total_len > 0) {
    memcpy(text, result_buf, total_len);
    *len_ptr = total_len;
} else {
    const char ok[] = "Done.\n";
    memcpy(text, ok, sizeof(ok) - 1);
    *len_ptr = sizeof(ok) - 1;
}

sem_post(sem_server);
}

sem_close(sem_child);
sem_close(sem_server);
munmap(shm_buf, SHM_SIZE);
close(shm_fd);
close(fd);

```

```
return 0;  
  
}
```

Протокол работы программы

```
mrnek@WindowsLaptop:~/Projects/MAI/MAI_OS_LABS/LAB3$ ./client a.txt  
260187: I'm parent, child PID = 260188  
Enter numbers (like '10 2 5'), blank line to exit.  
Input> 12 3 4 5 6  
Result: 12 / 3 = 4  
12 / 4 = 3  
12 / 5 = 2  
12 / 6 = 2  
Input> 23 98 38 2  
Result: 23 / 98 = 0  
23 / 38 = 0  
23 / 2 = 11  
Input>
```

Вывод

В ходе лабораторной работы я приобрел практические навыки при работе с разделяемой памятью, с ее отображением в адресное пространство процесса, а также потренировался в использовании семафоров.